REMOTE AI INFERENCE SERVER

Using Ubuntu Server, Ollama & Cloudflare Tunnel

Technical Deployment & Architecture Report

ABSTRACT

This report documents the end-to-end deployment of a remotely accessible Artificial Intelligence inference server built using entirely open-source and locally hosted components. The system leverages Ubuntu Server as the base operating system, Ollama as the local large language model inference engine, and Cloudflare Tunnel as the secure reverse-proxy mechanism for global accessibility. The project explores key areas of networking—including NAT traversal, protocol selection, reverse proxying, and DNS resolution—while also confronting real-world challenges such as firewall restrictions, API compatibility issues, and cross-platform command-line differences. The outcome is a functional, globally accessible AI server that operates without dependence on third-party cloud AI services, providing a cost-effective and privacy-respecting alternative architecture for AI inference deployment.

Keywords: *Ubuntu Server, Ollama, Cloudflare Tunnel, LLM Inference, Reverse Proxy, NAT Traversal, REST API, QUIC, HTTP/2, Remote Access, AI Deployment*

1. System Setup and Environment Configuration

The project began with the preparation of a bootable installation medium using Rufus, where an Ubuntu Server ISO image was written onto a USB drive with appropriate partition scheme settings. This ensured compatibility with the system firmware and allowed successful booting into the installation environment. Ubuntu Server was deliberately chosen over a desktop distribution due to its lightweight nature, absence of graphical overhead, and suitability for continuous

backend operations. During installation, essential parameters such as hostname, username, and password were configured, which play a critical role in system identification, authentication, and network communication.

Once installed, the system operated entirely in a terminal-based environment (TTY), requiring command-line interaction for all tasks. This provided deeper control over system operations and reduced unnecessary resource consumption. Post-installation, system packages were updated using the Advanced Package Tool (APT), ensuring that all dependencies, security patches, and repositories were synchronized with the latest versions. Network connectivity was verified using commands like ip a, which provided detailed information about active interfaces, IP addresses, and link status, confirming that the system was correctly connected to a local network.

2. Ollama Installation and AI Model Deployment

The next phase involved installing Ollama, a local large language model (LLM) inference engine, using a remote shell script fetched via curl. This method automated the installation process and ensured that all required binaries and dependencies were properly configured. Ollama was selected because it allows execution of advanced language models locally without relying on external APIs, thereby ensuring privacy and independence from cloud-based services.

The installation was performed using the following command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

After installation, the llama3 model was downloaded and executed locally. This demonstrated the ability to run AI inference directly on CPU hardware, even in the absence of a GPU. The model was tested using the CLI, which initially produced streaming outputs (token-by-token responses). Further exploration revealed that Ollama exposes a REST API on port 11434, enabling programmatic access. By sending structured JSON requests to this endpoint, the system could generate complete responses in a non-streaming format, making it suitable for integration with external applications.

3. Networking Concepts and Architecture

A significant portion of the project involved understanding and applying fundamental networking concepts. Initially, the Ollama service was accessible only via localhost (127.0.0.1), meaning it could only be accessed internally within the same machine. The system also operated within a private IP range (e.g., 10.x.x.x), which is typical for local area networks (LANs). These private IPs are not directly accessible over the internet due to Network Address Translation (NAT), which restricts inbound connections from external networks.

The project required exposing this local service globally without modifying router configurations or performing port forwarding. This introduced the concept of reverse proxy tunneling. Cloudflare Tunnel was used to establish a secure outbound connection from the local server to Cloudflare's edge network. This tunnel effectively mapped a public URL to the local service, allowing external users to access the AI server without directly exposing the machine to the internet.

Additionally, the project explored differences between communication protocols such as TCP and UDP. TCP provides reliable, connection-oriented communication, while UDP is faster but less reliable. Cloudflare Tunnel initially attempted to use QUIC (which operates over UDP), but due to network restrictions, this led to failures, requiring a switch to HTTP/2 over TCP.

---

4. Cloudflare Tunnel Implementation

Cloudflared was installed manually using a .deb package, especially after encountering issues with package repositories. A temporary (quick) tunnel was created using the following command:

```
cloudflared tunnel --url http://localhost:11434 --protocol http2
```

This command established a secure connection between the local server and Cloudflare's infrastructure, generating a public URL under the trycloudflare.com domain. This URL acted as an entry point for external users, forwarding requests to the local Ollama service. The tunnel operates as a reverse proxy, meaning that incoming requests from the internet are routed through Cloudflare and delivered to the local machine via the established connection.

The logs generated by cloudflared provided valuable insights into connection status, protocol usage, and error handling. It was observed that the tunnel remains active only as long as the terminal session is running, emphasizing its temporary nature. If the process is terminated or the system is shut down, the tunnel URL becomes invalid.

5. Problems Encountered and Solutions

Throughout the project, several real-world issues were encountered, each contributing to a deeper understanding of networking and system behavior. The following subsections detail each problem and its resolution.

5.1 QUIC Protocol Failure

One major issue was the failure of QUIC protocol connections, resulting in timeout errors. This occurred because many networks, especially institutional or campus WiFi, block UDP traffic. The solution involved explicitly forcing the tunnel to use HTTP/2, which operates over TCP and is universally supported.

5.2 DNS Resolution Errors (NXDOMAIN)

Another issue involved DNS resolution errors (NXDOMAIN), which occurred when the tunnel expired. Since quick tunnels are session-based, the generated domain ceases to exist once the process stops. Restarting the tunnel resolved this issue by generating a new URL.

5.3 APT Package Update Failures

The project also faced challenges with APT package updates, including hash mismatch errors. These were resolved by clearing cached package lists and re-fetching repository data:

```
sudo apt clean
sudo rm -rf /var/lib/apt/lists/*
sudo apt update
```

5.4 PowerShell curl Compatibility

On the client side, PowerShell introduced complications due to differences in command behavior. The curl command in PowerShell is an alias for Invoke-WebRequest, which does not support standard Linux-style flags. This required switching to curl.exe or using Invoke-RestMethod with proper JSON formatting.

5.5 403 Forbidden — Cloudflare Request Blocking

A critical issue was the 403 Forbidden error, where Cloudflare blocked API requests. This occurred because the requests lacked typical browser headers and were flagged as automated traffic. The issue was resolved by adding headers such as User-Agent, making the request appear as if it originated from a browser.

6. Security and System Design Considerations

One of the key advantages of using Cloudflare Tunnel is enhanced security. Since the server does not expose any open ports, it significantly reduces the attack surface. All incoming traffic is routed through Cloudflare's secure infrastructure, which acts as a protective layer between the client and the server.

The architecture follows a client-server model, where the client sends HTTP requests and the server processes them using the AI model. The use of REST APIs ensures compatibility with various platforms, including web applications, mobile apps, and backend systems.

However, the use of temporary tunnels introduces limitations such as lack of persistence and potential request blocking. For production environments, a named tunnel with a custom domain would provide a more stable and reliable solution.

7. Conclusion and Outcomes

The project successfully demonstrates the deployment of a globally accessible AI inference server using local hardware and open-source tools. By integrating Ubuntu Server, Ollama, and

Cloudflare Tunnel, the system enables remote access to a locally hosted language model without relying on external cloud services.

This implementation highlights practical applications of networking concepts such as NAT traversal, reverse proxying, protocol selection, and DNS resolution. It also showcases problem-solving skills in addressing real-world issues related to network restrictions, API communication, and system configuration.

Overall, the project serves as a comprehensive example of building a distributed system that combines local computation with global accessibility, offering a cost-effective and scalable alternative to traditional cloud-based AI solutions.

8. Protocol Behavior and Network Adaptation

During implementation, it was observed that Cloudflare Tunnel dynamically selects communication protocols, initially attempting QUIC for performance optimization. However, due to restrictive firewall policies blocking UDP traffic, the system required manual adaptation to HTTP/2 over TCP. This highlighted the importance of protocol flexibility in real-world deployments, where network environments vary significantly and cannot always support modern transport protocols.

9. Understanding the Reverse Proxy Mechanism

The Cloudflare Tunnel operates as a reverse proxy, where incoming client requests are first received by Cloudflare's edge servers and then securely forwarded to the local machine. This eliminates the need for exposing internal ports and ensures that the server remains hidden behind a secure intermediary layer, thereby improving both accessibility and security simultaneously.

10. Session-Based Tunnel Lifecycle

The temporary tunnel used in this project is inherently session-based, meaning its lifecycle is directly tied to the execution of the cloudflared process. Once the process is terminated or the system shuts down, the tunnel is destroyed, and the associated DNS entry becomes invalid. This behavior emphasizes the need for persistent services in production systems.

11. Impact of Network Environment on System Behavior

The project revealed that system behavior changes significantly across different networks. For example, tunnels failed in institutional networks due to strict firewall rules but functioned correctly in less restrictive home networks. This demonstrates the importance of environment-aware design and testing when deploying network-dependent systems.

12. Handling API Rate Limits and Request Filtering

Although not explicitly enforced in this setup, Cloudflare's infrastructure includes mechanisms for rate limiting and request filtering. The observed 403 errors indicate that certain patterns of requests were flagged as automated or suspicious. This highlights the importance of designing API interactions that mimic legitimate traffic patterns to avoid unintended blocking.

13. Command-Line Interface Limitations and Cross-Platform Differences

The project exposed differences between Linux and Windows command-line environments, particularly in handling HTTP requests. PowerShell's abstraction of curl led to compatibility issues, requiring the use of native commands or explicit executables. This emphasizes the need for platform-specific considerations when designing cross-environment workflows.

14. JSON Serialization and Data Formatting Challenges

A critical aspect of interacting with the Ollama API involved correctly formatting JSON payloads. Improper escaping of characters or incorrect structure resulted in failed requests. This highlights the importance of serialization standards in API communication, where even minor syntax errors can disrupt system functionality.

15. Service Dependency and Process Management

The system relies on multiple services running concurrently, including the Ollama server and the Cloudflare tunnel. If either service stops, the entire system becomes inaccessible. This demonstrates the importance of process management and service orchestration in maintaining system reliability and uptime.

16. Latency and Performance Considerations

Accessing the AI model remotely introduced additional latency compared to local execution. This delay is influenced by factors such as network speed, routing through Cloudflare's edge servers, and model inference time. Understanding these performance trade-offs is essential when designing systems that balance accessibility with responsiveness.

17. Scalability and Future Expansion Potential

The architecture designed in this project is inherently scalable. Additional models, services, or APIs can be deployed on the same server and exposed through separate tunnels or endpoints. This modular approach allows the system to evolve into a more complex distributed platform capable of supporting multiple users and applications simultaneously.

KEY TAKEAWAYS

- Ubuntu Server provides a lightweight, efficient foundation for backend AI deployments.
- Ollama enables fully local LLM inference without dependence on cloud AI APIs.
- Cloudflare Tunnel enables secure global access without exposing server ports.

- Protocol flexibility (QUIC vs HTTP/2) is critical in varied network environments.
- Session-based tunnels are suitable for testing; named tunnels are required for production.
- Cross-platform command-line differences (Linux vs Windows) must be accounted for in workflow design.
- Proper JSON formatting and request headers are essential for stable API communication.
- The modular architecture supports future expansion to multi-model or multi-user systems.

REFERENCES

[1] Cloudflare. (2024). Cloudflare Tunnel Documentation. https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/

[2] Ollama. (2024). Ollama Official Documentation. https://ollama.com/docs

[3] Canonical Ltd. (2024). Ubuntu Server Guide. https://ubuntu.com/server/docs

[4] Meta AI. (2024). LLaMA 3 Model Card. https://ai.meta.com/blog/meta-llama-3/

[5] Mozilla Developer Network. (2024). HTTP/2 Overview. https://developer.mozilla.org/en-US/docs/Glossary/HTTP_2

[6] IETF. (2021). RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport. https://datatracker.ietf.org/doc/html/rfc9000

[7] Microsoft. (2024). Invoke-WebRequest vs curl.exe in PowerShell. https://learn.microsoft.com/en-us/powershell/